HVACi PDF Extraction

Configurable, rule-based extraction into Excel

A practical guide and complete Python implementation for processing a folder of HVACi assessment PDFs, extracting fields using regex patterns or keywords, and consolidating the results into one Excel workbook.

No AI model or external API is required. Users add or update fields in a separate JSON configuration file; the Python program creates the corresponding Excel columns automatically.

Included in this guide

• Architecture and folder structure

• Installation and run instructions

• Configurable regex and keyword extraction rules

• Complete extraction_rules.json example

• Complete pdf_extractor.py source code

• Troubleshooting and quality-control notes

1. Overview

The program reads PDFs from an input folder, extracts text page by page using PyMuPDF, applies user-defined rules, and creates a consolidated Excel workbook. Each PDF becomes one row in the Extracted_Data sheet. Each configured field becomes a column. The QA_Log sheet records missing fields, warnings, and failed files.

Processing flow

Step	Description
1. Input	Place all source PDFs in the input folder. Subfolders can also be searched.
2. Text extraction	PyMuPDF reads text from each page. Optional OCR is attempted for pages with very little text.
3. Rules	Regex patterns and/or keywords from extraction_rules.json are applied to the extracted text.
4. Consolidation	The extracted values are stored as one row per PDF.
5. Excel output	A workbook is created with Extracted_Data and QA_Log sheets.
6. Troubleshooting	Extracted page text is saved under output/extracted_text.

2. Requirements and setup

Install Python packages in your terminal:

```
python -m pip install pymupdf pandas openpyxl
```

Create this structure in your project directory:

```
HVACi Project/
  pdf_extractor.py
  extraction_rules.json
  input/                # Place PDFs here
  output/               # Created automatically
    extracted_text/     # Extracted text files
    hvaci_extracted_data.xlsx
```

OCR note: scanned PDFs may need Tesseract installed and configured separately. If OCR is unavailable, the program records a warning; text-based PDFs can still be processed.

3. How to configure extraction rules

The JSON file contains a fields list. Each item defines an Excel column using a unique name and one or more regex patterns, or a list of keywords. For regex patterns, the first capturing group is returned. For keywords, the program returns text on the same line after the matching keyword.

Example: add a serial_number column

```
{
  "name": "serial_number",
  "patterns": [
    "\\bSerial\\s*(?:Number|No\\.?|#)\\s*:?\\s*([A-Z0-9-]+)"
  ]
}
```

Example: extract a technician name by keyword

```
{
  "name": "technician_name",
  "keywords": [
    "Technician Name",
    "Service Technician"
  ]
}
```

Add the new object inside the fields array in extraction_rules.json, separated from neighboring objects by commas. Keep field names unique and maintain valid JSON syntax. Then rerun the Python script; the column is created automatically without changing the Python source.

4. Complete extraction_rules.json

Copy the following content into a file named extraction_rules.json. The patterns are starting examples; adjust them after comparing the extracted text with your actual report layouts.

```
{
  "fields": [
    {
      "name": "claim_number",
      "patterns": [
        "\\bClaim\\s*(?:Number|No\\.?|#)\\s*:?\\s*([A-Z0-9-]+)"
      ]
    },
    {
      "name": "hvaci_file_number",
      "patterns": [
        "\\bHVACi\\s*(?:File\\s*)?(?:Number|No\\.?|#)\\s*:?\\s*([A-Z0-9-]+)"
      ]
    },
    {
      "name": "report_date",
      "patterns": [
        "\\bReport\\s*Date\\s*:?\\s*(\\d{1,2}[/-]\\d{1,2}[/-]\\d{4})"
      ]
    },
    {
      "name": "manufacturer",
      "patterns": [
        "\\bManufacturer\\s*:?\\s*([^\\r\\n]+)",
        "\\bBrand\\s*:?\\s*([^\\r\\n]+)"
      ]
    },
    {
      "name": "equipment_model",
      "patterns": [
        "\\b(?:Equipment\\s*Model|Model\\s*Number|Model\\s*No\\.?|Model\\s*#)\\s*:?\\s*([A-Z0-9-]+)"
      ]
    },
    {
      "name": "model_year",
      "patterns": [
        "\\b(?:Approx(?:imate)?\\.?\\s*)?(?:Model\\s*)?Year\\s*:?\\s*((?:19|20)\\d{2})"
      ]
    },
    {
      "name": "cause_of_loss",
      "patterns": [
        "\\bCause\\s*of\\s*Loss\\s*:?\\s*([^\\r\\n]+)"
      ]
    },
    {
      "name": "repair_recommendation",
      "patterns": [
        "\\b(?:Repair\\s*Recommendation|Recommended\\s*Repair|Recommendation)\\s*:?\\s*([^\\r\\n]+)"
      ]
    },
    {
      "name": "total_repair_cost",
      "patterns": [
        "\\bTotal\\s*Recommended\\s*Repair\\s*Cost\\s*:?\\s*\\$?\\s*([\\d,]+\\.\\d{2})",
        "\\bTotal\\s*Repair\\s*Cost\\s*:?\\s*\\$?\\s*([\\d,]+\\.\\d{2})"
      ]
    },
    {
      "name": "depreciation",
      "patterns": [
        "\\bDepreciation\\s*:?\\s*\\$?\\s*([\\d,]+\\.\\d{2})"
      ]
    },
    {
      "name": "recommended_acv",
      "patterns": [
        "\\bRecommended\\s*ACV\\s*:?\\s*\\$?\\s*([\\d,]+\\.\\d{2})",
        "\\bActual\\s*Cash\\s*Value\\s*:?\\s*\\$?\\s*([\\d,]+\\.\\d{2})"
      ]
    }
  ]
}
```

5. Complete Python program: pdf_extractor.py

Copy the complete code below into pdf_extractor.py. It processes all PDFs in input, continues after individual file failures, and writes the consolidated workbook and extracted text files.

```
import json
import re
from pathlib import Path

import fitz
import pandas as pd


# ============================================================
# 1. CONFIGURATION
# ============================================================

BASE_DIR = Path(__file__).resolve().parent

INPUT_DIR = BASE_DIR / "input"
OUTPUT_DIR = BASE_DIR / "output"
RULES_FILE = BASE_DIR / "extraction_rules.json"

EXCEL_FILE = OUTPUT_DIR / "hvaci_extracted_data.xlsx"
TEXT_DIR = OUTPUT_DIR / "extracted_text"

# Try OCR when a page contains very little text.
OCR_THRESHOLD = 40

# True means search input subfolders as well.
INCLUDE_SUBFOLDERS = True


# ============================================================
# 2. READ USER-DEFINED EXTRACTION RULES
# ============================================================

def load_rules():
    if not RULES_FILE.exists():
        raise FileNotFoundError(
            f"Rules file not found: {RULES_FILE}"
        )

    with open(RULES_FILE, "r", encoding="utf-8") as file:
        config = json.load(file)

    fields = config.get("fields", [])

    if not fields:
        raise ValueError(
            "No extraction fields found in extraction_rules.json"
        )

    names = [field.get("name") for field in fields]

    if any(not name for name in names):
        raise ValueError(
            "Every configured field must have a non-empty name."
        )

    if len(names) != len(set(names)):
        raise ValueError(
            "Each configured field name must be unique."
        )

    return fields


# ============================================================
# 3. EXTRACT TEXT FROM A PDF
# ============================================================

def extract_pdf_text(pdf_path):
    page_texts = []
    warnings = []

    with fitz.open(pdf_path) as document:
        page_count = len(document)

        for page_index, page in enumerate(document):

            page_number = page_index + 1

            text = page.get_text("text", sort=True)
            method = "Text extraction"

            # OCR fallback for scanned or image-heavy pages.
            if len(text.strip()) < OCR_THRESHOLD:
```

```
                try:
                    text_page = page.get_textpage_ocr(
                        language="eng",
                        dpi=200,
                        full=True
                    )

                    ocr_text = page.get_text(
                        "text",
                        textpage=text_page,
                        sort=True
                    )

                    if len(ocr_text.strip()) > len(text.strip()):
                        text = ocr_text
                        method = "OCR"

                except Exception as error:
                    warnings.append(
                        f"Page {page_number}: OCR failed or is "
                        f"not installed: {error}"
                    )

            text = text.replace("\xa0", " ")
            text = re.sub(r"[ \t]+", " ", text)
            text = text.strip()

            if not text:
                warnings.append(
                    f"Page {page_number}: no text extracted."
                )

            page_texts.append({
                "page_number": page_number,
                "method": method,
                "text": text
            })

    return page_count, page_texts, warnings


# ============================================================
# 4. EXTRACT A FIELD USING REGEX OR KEYWORDS
# ============================================================

def extract_field(field_config, full_text):
    """
    Supported rule styles:

    1. Regex:
       {
         "name": "claim_number",
         "patterns": ["Claim\\s*#?\\s*:?\\s*([A-Z0-9-]+)"]
       }

    2. Keyword:
       {
         "name": "technician_name",
         "keywords": ["Technician Name", "Service Technician"]
       }

    For regex rules, the first capturing group is returned.
    For keyword rules, text on the same line after the keyword
    is returned.
    """

    # Method A: regular expressions.
    patterns = field_config.get("patterns", [])

    if isinstance(patterns, str):
        patterns = [patterns]

    for pattern in patterns:
        try:
            match = re.search(
                pattern,
                full_text,
                flags=re.IGNORECASE | re.MULTILINE
            )

            if match:
                # Prefer group 1 when it exists.
                if match.lastindex:
                    value = match.group(1)
                else:
                    value = match.group(0)

                if value and value.strip():
                    return clean_value(value)

        except re.error as error:
```

```
            raise ValueError(
                f"Invalid regex for field "
                f"'{field_config.get('name')}': {error}"
            ) from error

    # Method B: keyword-based extraction.
    keywords = field_config.get("keywords", [])

    if isinstance(keywords, str):
        keywords = [keywords]

    for keyword in keywords:
        for line in full_text.splitlines():
            match = re.search(
                re.escape(keyword),
                line,
                flags=re.IGNORECASE
            )

            if not match:
                continue

            # Extract text after the keyword on the same line.
            value = line[match.end():]

            value = re.sub(
                r"^\s*[:=\-–—#]+\s*",
                "",
                value
            ).strip()

            if value:
                return clean_value(value)

    return None


def clean_value(value):
    value = str(value).strip()
    value = re.sub(r"\s+", " ", value)

    return value if value else None


# ============================================================
# 5. FIND ALL PDFs
# ============================================================

def find_pdfs():
    if not INPUT_DIR.exists():
        raise FileNotFoundError(
            f"Input folder does not exist: {INPUT_DIR}"

        )

    if INCLUDE_SUBFOLDERS:
        pdf_files = INPUT_DIR.rglob("*")
    else:
        pdf_files = INPUT_DIR.glob("*")

    return sorted(
        path for path in pdf_files
        if path.is_file() and path.suffix.lower() == ".pdf"
    )


# ============================================================
# 6. PROCESS ONE PDF
# ============================================================

def process_pdf(pdf_path, fields):
    page_count, page_records, warnings = extract_pdf_text(
        pdf_path
    )

    full_text = "\n\n".join(
        f"--- PAGE {record['page_number']} ---\n"
        f"{record['text']}"
        for record in page_records
    )

    result = {
        "source_pdf": str(pdf_path.relative_to(INPUT_DIR)),
        "pages_processed": page_count
    }

    missing_fields = []

    # Each configured field automatically becomes an Excel column.
    for field_config in fields:
```

```
        field_name = field_config["name"]

        try:
            value = extract_field(field_config, full_text)
        except Exception as error:
            value = None
            warnings.append(str(error))

        result[field_name] = value

        if value is None:
            missing_fields.append(field_name)

    # Save extracted text separately for troubleshooting.
    TEXT_DIR.mkdir(parents=True, exist_ok=True)

    relative_path = pdf_path.relative_to(INPUT_DIR)
    text_filename = "__".join(relative_path.parts)
    text_filename = re.sub(
        r'[<>:"/\\|?*]',
        "_",
        text_filename
    )

    text_path = TEXT_DIR / f"{text_filename}.txt"

    text_path.write_text(
        full_text,
        encoding="utf-8"
    )

    # Record quality checks.
    qa = {
        "source_pdf": str(relative_path),
        "status": (
            "REVIEW" if missing_fields or warnings else "OK"
        ),
        "missing_fields": ", ".join(missing_fields),

        "warnings": " | ".join(warnings),
        "extracted_text_file": str(text_path)
    }

    return result, qa


# ============================================================
# 7. EXPORT RESULTS TO EXCEL
# ============================================================

def save_excel(results, qa_results):
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

    data_df = pd.DataFrame(results)
    qa_df = pd.DataFrame(qa_results)

    with pd.ExcelWriter(
        EXCEL_FILE,
        engine="openpyxl"
    ) as writer:

        data_df.to_excel(
            writer,
            sheet_name="Extracted_Data",
            index=False
        )

        qa_df.to_excel(
            writer,
            sheet_name="QA_Log",
            index=False
        )

        for worksheet in writer.book.worksheets:
            worksheet.freeze_panes = "A2"
            worksheet.auto_filter.ref = worksheet.dimensions

            # Keep long text columns readable.
            for cells in worksheet.columns:
                header = str(cells[0].value or "")
                column_letter = cells[0].column_letter

                if header in [
                    "warnings",
                    "missing_fields",
                    "extracted_text_file"
                ]:
                    width = 45
                else:
                    max_length = max(
```

```
                        len(str(cell.value or ""))
                        for cell in cells
                    )
                    width = min(max(max_length + 2, 14), 35)

                worksheet.column_dimensions[column_letter].width = width

    print(f"\nExcel workbook created:\n{EXCEL_FILE}")


# ============================================================
# 8. MAIN PROGRAM
# ============================================================

def main():
    INPUT_DIR.mkdir(parents=True, exist_ok=True)
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

    fields = load_rules()
    pdf_files = find_pdfs()

    if not pdf_files:
        print(f"No PDF files found in:\n{INPUT_DIR}")
        print("Place your PDFs in the input folder and try again.")

        return

    print(f"PDFs found: {len(pdf_files)}")
    print(f"Configured columns: {len(fields)}")
    print("Starting extraction...\n")

    results = []
    qa_results = []

    for index, pdf_path in enumerate(pdf_files, start=1):
        print(
            f"[{index}/{len(pdf_files)}] "
            f"Processing: {pdf_path.name}"
        )

        try:
            result, qa = process_pdf(pdf_path, fields)

        except Exception as error:
            # Continue processing other PDFs if one fails.
            result = {
                "source_pdf": str(pdf_path.relative_to(INPUT_DIR)),
                "pages_processed": None
            }

            for field_config in fields:
                result[field_config["name"]] = None

            qa = {
                "source_pdf": str(pdf_path.relative_to(INPUT_DIR)),
                "status": "FAILED",
                "missing_fields": ", ".join(
                    field["name"] for field in fields
                ),
                "warnings": str(error),
                "extracted_text_file": None
            }

        results.append(result)
        qa_results.append(qa)

    save_excel(results, qa_results)

    successful = sum(
        1 for row in qa_results
        if row["status"] != "FAILED"
    )

    failed = sum(
        1 for row in qa_results
        if row["status"] == "FAILED"
    )

    review = sum(
        1 for row in qa_results
        if row["status"] == "REVIEW"
    )

    print("\n========== SUMMARY ==========")
    print(f"PDFs found:       {len(pdf_files)}")
    print(f"Processed:        {successful}")
    print(f"Failed:           {failed}")
    print(f"Needs review:     {review}")
    print(f"Excel output:     {EXCEL_FILE}")

if __name__ == "__main__":
```

```
main()
```

6. Run the program

1. Save both files in the project folder.

2. Place your PDFs in the input folder.

3. Open a terminal in the project folder and run:

```
python pdf_extractor.py
```

The workbook will be saved to output/hvaci_extracted_data.xlsx. The output folder is created automatically.

7. Output workbook

Sheet / folder	Contents
Extracted_Data	One row per PDF. Columns include source_pdf, pages_processed, and every configured field.
QA_Log	Processing status, missing fields, warnings, and the extracted-text file location.
output/extracted_text	One .txt file per PDF containing the extracted text and page markers.

8. Troubleshooting and quality checks

PDF not found

Make sure the PDFs are inside the input folder. The program searches subfolders by default.

Fields are blank

Open the corresponding file in output/extracted_text and adjust the regex or keyword to match the exact text.

Scanned PDF

Install Tesseract OCR and ensure PyMuPDF can access it. Check the QA_Log for OCR warnings.

Regex errors

Validate JSON escaping and regex syntax. In JSON, backslashes are doubled, for example \\s.

Different layouts

Add multiple patterns for a field. The first matching pattern wins.

Amounts in tables

Text extraction order can be imperfect for complex tables. Verify costs against the source PDF.

Duplicate filenames

Text filenames are based on relative input paths, though very long or unusual paths should still be tested.

9. Recommended pilot before processing thousands of files

• Start with 10-20 representative PDFs, including different report layouts and scanned pages.

• Compare each extracted row with the original report and record incorrect or missing fields.

• Improve the JSON patterns before running the full batch.

• Review all QA_Log rows marked REVIEW or FAILED.

• Treat extracted values as candidates until the extraction rules have been verified.

Privacy: process company claim documents only in accordance with your organization's data-handling rules. This implementation uses local Python libraries and does not call an AI API.